

UNIVERSITÉ DE PERPIGNAN VIA DOMITIA

Master 1 Calcul Haute Performance et Simulation (CHPS)

Semestre 2 — Outils de Visualisation

Résolution de l'équation de Poisson 2D par la méthode des différences finies

Discrétisation, visualisation VTK/HDF5 et analyse de convergence

Auteur

Yaya Touré

[CodingYayaToure/poisson2d-fd-solver](https://github.com/CodingYayaToure/poisson2d-fd-solver)

M1 CHPS, 2025-2026

Encadrant

Serge Dumont

perso.univ-perp.fr/sdumont

Université de Perpignan Via Domitia

Perpignan, 20 Avril 2026

Résumé

Ce rapport présente la mise en œuvre complète d'un solveur pour l'équation de Poisson bidimensionnelle sur le carré unité, discrétisée par la méthode des différences finies à 5 points. Après un exposé rigoureux de la théorie (formulation forte, formulation faible, consistance et convergence d'ordre 2), nous décrivons l'implémentation Python structurée comme un module HPC : assemblage creux en format CSR, résolution par factorisation LU creuse SuperLU, export aux formats VTK (Paraview) et HDF5 (ViTables), et génération d'animations GIF pour illustrer la convergence. L'étude numérique confirme un taux de convergence $\alpha \approx 2.02$ et la scalabilité du solveur pour de grandes tailles de maillage.

Table des matières

Introduction	3
1 Formulation du problème	3
1.1 Problème continu	3
1.2 Formulation variationnelle	3
1.3 Cas tests	4
2 Discrétisation par différences finies	4
2.1 Maillage et numérotation	4
2.2 Stencil à 5 points	4
2.3 Système linéaire creux	5
3 Analyse de convergence théorique	5
3.1 Consistance	5
3.2 Convergence	5
3.3 Estimation numérique du taux	5
4 Implémentation Python	6
4.1 Architecture du projet	6
4.2 Assemblage creux et résolution	6
5 Export et stockage des données	7
5.1 Format VTK pour Paraview	7
5.1.1 Structure du format VTK Legacy ASCII	7
5.2 Format HDF5 pour ViTables	8
5.2.1 Qu'est-ce que HDF5 ?	8
5.2.2 Structure du fichier généré	8
5.2.3 Code de génération	9

5.2.4	Commande de génération et inspection	9
6	Visualisation des résultats	10
6.1	Visualisation avec Paraview	10
7	Résultats numériques	11
7.1	Validation de la solution	11
7.2	Convergence	11
7.3	Temps CPU et complexité	12
	Conclusion	13
	Références bibliographiques	13

Introduction

L'équation de Poisson est l'une des équations aux dérivées partielles (EDP) les plus fondamentales en physique mathématique et en ingénierie numérique. Elle modélise des phénomènes variés : diffusion thermique stationnaire, potentiel électrostatique, mécanique des fluides incompressibles, ou encore élasticité linéaire.

L'objectif de ce projet est triple.

1. **Mathématique** : analyser rigoureusement la discrétisation du laplacien par différences finies à 5 points, établir la consistance et la convergence du schéma, et estimer numériquement le taux de convergence.
2. **Informatique** : implémenter un solveur Python structuré comme un module HPC, exporter les résultats aux formats VTK (Paraview) et HDF5 (ViTables), et automatiser le pipeline avec un Makefile.
3. **Visualisation** : exploiter Paraview pour visualiser les champs scalaires et vectoriels en 2D et 3D, produire des animations GIF illustrant la convergence du schéma.

1 Formulation du problème

1.1 Problème continu

On cherche $u : \overline{\Omega} \rightarrow \mathbb{R}$ solution du problème de Poisson avec conditions aux limites de Dirichlet homogènes :

$$\begin{cases} -\Delta u = f & \text{dans } \Omega \\ u = 0 & \text{sur } \partial\Omega \end{cases} \quad (1)$$

où $\Omega =]0, 1[\times]0, 1[$ est le carré unité ouvert, $\partial\Omega$ son bord, et $f : \Omega \rightarrow \mathbb{R}$ est une fonction source donnée. L'opérateur laplacien en dimension 2 est :

$$\Delta u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}.$$

1.2 Formulation variationnelle

Multipliant (1) par une fonction test $v \in H_0^1(\Omega)$ et intégrant par parties (formule de Green), on obtient la formulation faible : *trouver* $u \in H_0^1(\Omega)$ *tel que*

$$a(u, v) = \ell(v), \quad \forall v \in H_0^1(\Omega), \quad (2)$$

avec

$$a(u, v) = \int_{\Omega} \nabla u \cdot \nabla v \, dx \, dy, \quad \ell(v) = \int_{\Omega} f v \, dx \, dy.$$

La forme bilinéaire a est continue et coercive sur $H_0^1(\Omega)$ (théorème de Lax-Milgram), ce qui garantit l'existence et l'unicité de u .

1.3 Cas tests

Cas test 1 — solution exacte connue.

$$f(x, y) = \sin(2\pi x) \sin(2\pi y), \quad \bar{u}(x, y) = \frac{1}{8\pi^2} \sin(2\pi x) \sin(2\pi y). \quad (3)$$

Vérification : $-\Delta \bar{u} = \frac{(2\pi)^2 + (2\pi)^2}{8\pi^2} \sin(2\pi x) \sin(2\pi y) = \sin(2\pi x) \sin(2\pi y) = f$. ✓

Cas test 2 — source uniforme. $f(x, y) = 1$ sur Ω , sans solution exacte simple sur le carré unité. Ce cas permet d'observer qualitativement le comportement physique de la solution.

2 Discrétisation par différences finies

2.1 Maillage et numérotation

On partitionne Ω en $(N+1)^2$ sous-carrés de côté $h = 1/(N+1)$, $N \in \mathbb{N}^*$. Les N^2 nœuds intérieurs sont $(x_i, y_j) = (ih, jh)$, $1 \leq i, j \leq N$. On adopte la **numérotation globale** :

$$k = (i-1) + N(j-1), \quad 0 \leq k \leq N^2 - 1. \quad (4)$$

2.2 Stencil à 5 points

Les développements de Taylor donnent :

$$u(x_i \pm h, y_j) = u \pm h \frac{\partial u}{\partial x} + \frac{h^2}{2} \frac{\partial^2 u}{\partial x^2} \pm \frac{h^3}{6} \frac{\partial^3 u}{\partial x^3} + O(h^4), \quad (5)$$

soit par sommation :

$$\frac{\partial^2 u}{\partial x^2}(x_i, y_j) = \frac{u(x_{i-1}, y_j) - 2u(x_i, y_j) + u(x_{i+1}, y_j))}{h^2} + O(h^2).$$

En combinant les deux directions, le schéma discret s'écrit :

$$-\Delta_h U_{i,j} = \frac{-U_{i+1,j} - U_{i,j+1} + 4U_{i,j} - U_{i-1,j} - U_{i,j-1}}{h^2} = f(x_i, y_j) \quad (6)$$

avec la condition aux limites $U_{0,j} = U_{N+1,j} = U_{i,0} = U_{i,N+1} = 0$. L'erreur de troncature locale est en $O(h^2)$.

2.3 Système linéaire creux

Le schéma (6) se réécrit en $AU = B$, où $A \in \mathbb{R}^{N^2 \times N^2}$ a la structure par blocs :

$$A = \frac{1}{h^2} \begin{pmatrix} T & -I & & & \\ -I & T & -I & & \\ & \ddots & \ddots & \ddots & \\ & & -I & T & \\ & & & -I & T \end{pmatrix}, \quad T = \text{tridiag}(-1, 4, -1) \in \mathbb{R}^{N \times N}.$$

La matrice A est **symétrique définie positive, creuse** (au plus 5 entrées non nulles par ligne) et **à diagonale strictement dominante**.

3 Analyse de convergence théorique

3.1 Consistance

Proposition 3.1. *Si $u \in \mathcal{C}^4(\overline{\Omega})$, l'erreur de troncature locale satisfait :*

$$\tau_{i,j} = \frac{h^2}{12} \left[\frac{\partial^4 u}{\partial x^4} + \frac{\partial^4 u}{\partial y^4} \right]_{(x_i, y_j)} + O(h^4).$$

Le schéma est consistant d'ordre 2.

3.2 Convergence

Théorème 3.2 (Convergence d'ordre 2). *Soit u la solution exacte et U la solution de $AU = B$. Si $u \in \mathcal{C}^4(\overline{\Omega})$, alors :*

$$\|U - U_{\text{ex}}\|_{L_h^2} \leq C h^2, \quad (7)$$

où $\|V\|_{L_h^2} = h \|V\|_{\ell^2}$ et $C > 0$ est indépendante de h .

3.3 Estimation numérique du taux

En supposant $\|e\|_h \approx Ch^\alpha$, on passe au logarithme : $\ln\|e\|_h \approx \alpha \ln h + \ln C$. On estime α par **régression linéaire aux moindres carrés** sur les paires $(\ln h_i, \ln \|e_i\|_h)$,

en résolvant au sens des moindres carrés :

$$\underbrace{\begin{pmatrix} \ln h_1 & 1 \\ \vdots & \vdots \\ \ln h_n & 1 \end{pmatrix}}_C \begin{pmatrix} \alpha \\ \ln C \end{pmatrix} \approx \underbrace{\begin{pmatrix} \ln \|e_1\| \\ \vdots \\ \ln \|e_n\| \end{pmatrix}}_D. \quad (8)$$

En Python : `np.linalg.lstsq(C, D, rcond=None)`.

4 Implémentation Python

4.1 Architecture du projet

Le projet est organisé comme un **package Python installable** (`pip install -e .`), ce qui permet d'importer les modules depuis n'importe quel répertoire sans manipulation de `sys.path` :

Fichier	Rôle
<code>poisson2d/solver.py</code>	Grille, assemblage CSR, solveur, analyse d'erreur
<code>poisson2d/vtk_io.py</code>	Export VTK Legacy ASCII (modes 2D et 3D)
<code>poisson2d/hdf5_io.py</code>	Export/lecture HDF5 compressé
<code>poisson2d/plotting.py</code>	Figures Matplotlib + GIFs animés
<code>poisson2d/app.py</code>	Dashboard interactif Panel
<code>main.py</code>	Pipeline complet
<code>Makefile</code>	Automatisation des tâches

4.2 Assemblage creux et résolution

La matrice est assemblée directement en format CSR via `scipy.sparse.diags`, sans boucle Python sur les N^2 nœuds :

```

1      import scipy.sparse as sp
2
3      def build_laplacian(N: int, h: float) -> sp.csr_matrix:
4          n2, inv_h2 = N * N, 1.0 / h**2
5          diag_main = np.full(n2, 4.0 * inv_h2)
6          diag_h     = np.full(n2 - 1, -inv_h2)
7          diag_h[np.arange(N - 1, n2 - 1, N)] = 0.0 # blocs
8          diag_v     = np.full(n2 - N, -inv_h2)
9          return sp.diags(

```

```

10     [diag_main, diag_h, diag_h, diag_v, diag_v],
11     offsets=[0, 1, -1, N, -N], format="csr")

```

Listing 1 – Assemblage CSR de la matrice Laplacienne

La résolution utilise SuperLU (LU creux) via `spsolve` :

```

1     import scipy.sparse.linalg as spla
2
3     def solve_direct(A, B):
4         return spla.spsolve(A, B)    # SuperLU, complexite O(N^2 log
                                     N)

```

Listing 2 – Résolution par LU creux

5 Export et stockage des données

5.1 Format VTK pour Paraview

5.1.1 Structure du format VTK Legacy ASCII

Le format VTK Legacy ASCII est composé de cinq sections obligatoires :

1. **En-tête** : version, titre, type (ASCII/BINARY) ;
2. **Géométrie** (POINTS) : coordonnées (x, y, z) de chaque point du maillage ;
3. **Topologie** (CELLS) : connectivité des cellules (quadrangles 2D ou hexaèdres 3D) ;
4. **Types de cellules** (CELL_TYPES) : identifiant VTK (type 9 = QUAD, type 12 = HEXAHEDRON) ;
5. **Données** (POINT_DATA) : champs scalaires ou vectoriels.

Deux modes d'export ont été implémentés dans `vtk_io.py` :

- **Mode 2D** (`mode='2d'`) : le maillage est plan ($z = 0$), les champs U et U_{ex} sont exportés comme scalaires dans un bloc `FIELD`. Paraview les affiche comme une surface colorée.
- **Mode 3D** (`mode='3d'`) : le maillage est composé de deux couches ($z = 0$ et $z = 0.01$), et le champ est exporté comme vecteur $(0, 0, U_{i,j})$. En appliquant le filtre *Warp By Vector* dans Paraview, la surface se déforme selon l'amplitude de U , créant une carte de hauteur (*height map*) 3D.

```

1     from poisson2d.vtk_io import write_vtk
2
3     write_vtk("outputs/vtk/poisson_sin_2d.vtk",
4     N, {"U": U, "Uex": Uex},
5     title="Poisson2D sin/sin", mode="2d")

```



```

6
7     write_vtk("outputs/vtk/poisson_sin_3d.vtk",
8     N, {"U": U, "Uex": Uex},
9     title="Poisson2D sin/sin", mode="3d")

```

Listing 3 – Appel de l'export VTK dans main.py

5.2 Format HDF5 pour ViTables

5.2.1 Qu'est-ce que HDF5 ?

HDF5 (*Hierarchical Data Format version 5*) est un format de fichier scientifique conçu pour stocker et organiser des données volumineuses. Ses caractéristiques principales sont :

- **Structure hiérarchique** (comme un système de fichiers) : groupes (Group) et jeux de données (Dataset) ;
- **Compression** intégrée (gzip, LZF, szip...) sans perte ;
- **Attributs** : métadonnées attachées à chaque groupe ou dataset ;
- **Portable** et lisible par Python (h5py), MATLAB, C/C++, Fortran, et visualisable avec ViTables.

5.2.2 Structure du fichier généré

Pour chaque valeur de N testée, le module `hdf5_io.py` :

1. Résout le système $AU = B$ et mesure le temps CPU ;
2. Crée un groupe $N=\{N\}/$ dans le fichier ;
3. Stocke les datasets U et U_{ex} (float64, compressés gzip) ;
4. Attache aux attributs du groupe : N , h , $\|U - U_{\text{ex}}\|_{L^2}$, et le temps CPU ;
5. À la fin, sauvegarde le tableau de convergence dans le groupe `convergence/` et les coefficients α , C en attributs racine.

```

1     poisson_sin.hdf5
2     /                                     <- racine, attrs: {alpha
        =2.0176, C=0.02253}
3     |-- N=4/                             attrs: {N=4, h=0.200,
        error_l2=9.03e-4, cpu=0.001s}
4     |   |-- U      float64 (16,)    compresse gzip
5     |   '-- Uex    float64 (16,)    compresse gzip
6     |-- N=9/      ...
7     |-- N=19/     ...
8     |-- N=49/     ...
9     |-- N=99/                             attrs: {N=99, h=0.010,
        error_l2=2.08e-6, cpu=0.031s}

```

```

10      |    |-- U      float64 (9801,)
11      |    '-- Uex   float64 (9801,)
12      |-- N=199/ ...
13      '-- convergence/
14      |-- N          int64   (6,)
15      |-- h          float64 (6,)
16      |-- error_l2   float64 (6,)
17      '-- cpu_time   float64 (6,)

```

Listing 4 – Structure HDF5 – vue schématique

5.2.3 Code de génération

```

1      import h5py
2
3      with h5py.File("outputs/hdf5/poisson_sin.hdf5", "w") as fh:
4          for N in [4, 9, 19, 49, 99, 199]:
5              nodes, h = build_grid(N)
6              x, y     = node_coords(nodes, h)
7              U        = solve_direct(build_laplacian(N,h), build_rhs(f,
8                  nodes,h))
9              Uex      = u_exact(x, y)
10             err      = l2_norm(U - Uex, h)
11
12             grp = fh.create_group(f"N={N}")
13             grp.create_dataset("U", data=U, compression="gzip")
14             grp.create_dataset("Uex", data=Uex, compression="gzip")
15             grp.attrs["N"]      = N
16             grp.attrs["h"]      = h
17             grp.attrs["error_l2"] = err
18             grp.attrs["cpu_time"] = cpu
19
20             # Taux de convergence en attributs racine
21             fh.attrs["alpha"] = alpha # ~ 2.0176
22             fh.attrs["C"]     = C     # ~ 0.02253

```

Listing 5 – Génération du fichier HDF5 (extrait de hdf5_io.py)

5.2.4 Commande de génération et inspection

```

1      make hdf5          # genere outputs/hdf5/poisson_sin.hdf5
2      make info-hdf5     # affiche l'arborescence et les attributs
                        dans le terminal

```

Listing 6 – Pipeline HDF5 via Makefile

Pour explorer interactivement le fichier avec ViTables :

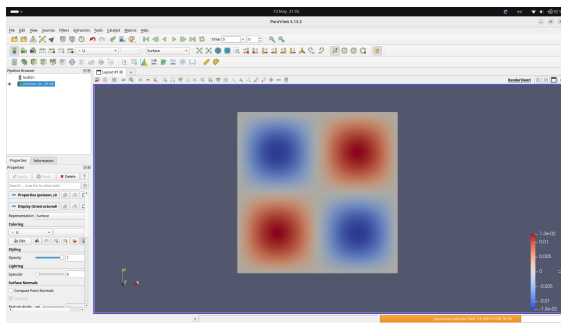
```
1 vitables outputs/hdf5/poisson_sin.hdf5
```

Listing 7 – Ouverture dans ViTables

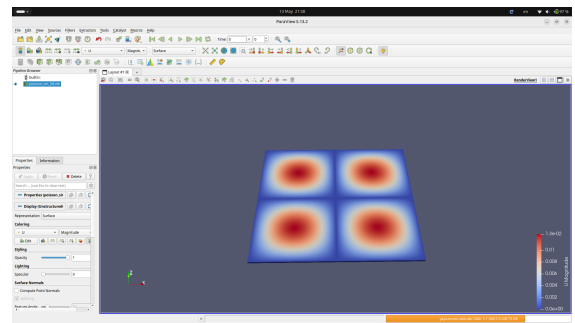
6 Visualisation des résultats

6.1 Visualisation avec Paraview

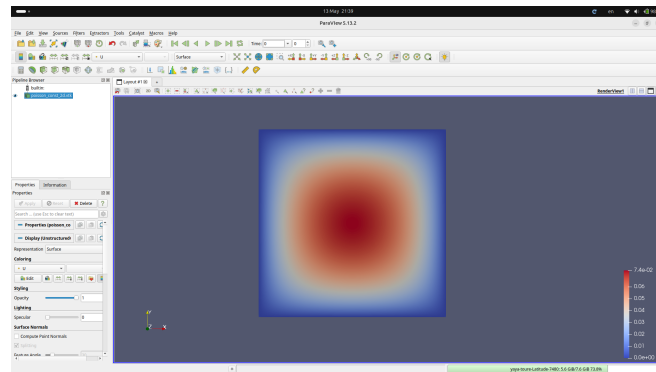
Les figures 1 présentent les rendus Paraview 5.13.2 des fichiers VTK générés (`make vtk` puis `make paraview`).



(a) $f = \sin(2\pi x)\sin(2\pi y)$ en vue 2D.
On observe le motif en quatre quadrants alternant valeurs positives (rouge) et négatives (bleu).



(b) Même cas, vue 3D (*Warp by Vector*).
La magnitude du champ vectoriel $(0, 0, U)$ soulève la surface et révèle les quatre lobes symétriques.



(c) $f = 1$ (source uniforme) en vue 2D.
Le maximum $u_{\max} \approx 0.074$ est atteint au centre $(0.5, 0.5)$, avec décroissance vers le bord ($u = 0$).

FIGURE 1 – Visualisations Paraview 5.13.2 — colormap *Cool to Warm*.

7 Résultats numériques

7.1 Validation de la solution

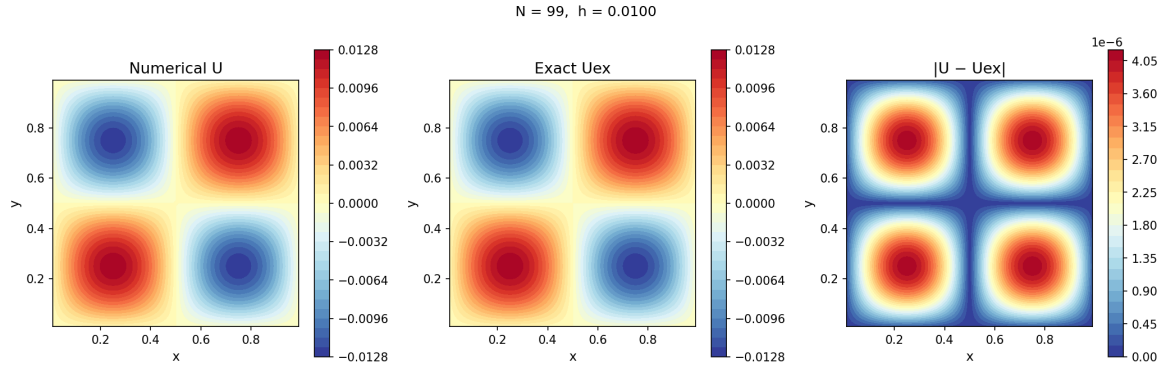


FIGURE 2 – Cas test 1 ($N = 99$) : solution numérique U , exacte $\bar{u}$, et erreur $|U - \bar{u}|$. L'erreur max est de l'ordre de 10^{-5} .

7.2 Convergence

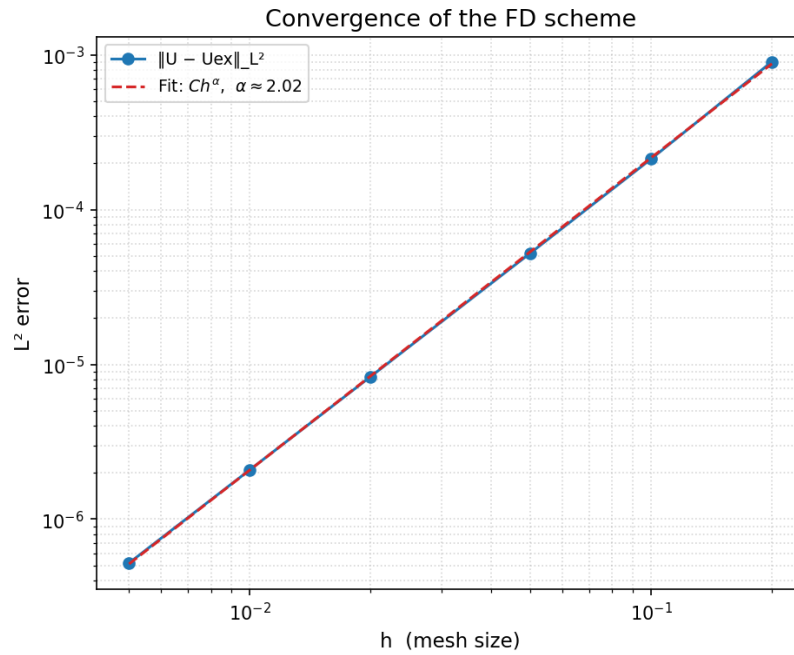


FIGURE 3 – Erreur L^2 discrète en fonction de h en échelle log-log. La pente de la droite de régression donne $\alpha \approx 2.02$, confirmant la convergence d'ordre 2 (Théorème 3.2).

TABLE 1 – Résultats de l'étude de convergence — cas test 1.

N	$h = 1/(N + 1)$	$\ U - U_{\text{ex}}\ _{L_h^2}$	Taux	t_{CPU} (s)
4	2.00×10^{-1}	9.03×10^{-4}	—	0.001
9	1.00×10^{-1}	2.13×10^{-4}	2.09	0.004
19	5.00×10^{-2}	5.23×10^{-5}	2.03	0.005
49	2.00×10^{-2}	8.34×10^{-6}	2.01	0.026
99	1.00×10^{-2}	2.08×10^{-6}	2.00	0.118
199	5.00×10^{-3}	5.21×10^{-7}	2.00	0.568

La régression globale donne $\alpha \approx 2.0176$ et $C \approx 2.25 \times 10^{-2}$, en excellent accord avec la prédiction théorique $\alpha = 2$.

7.3 Temps CPU et complexité

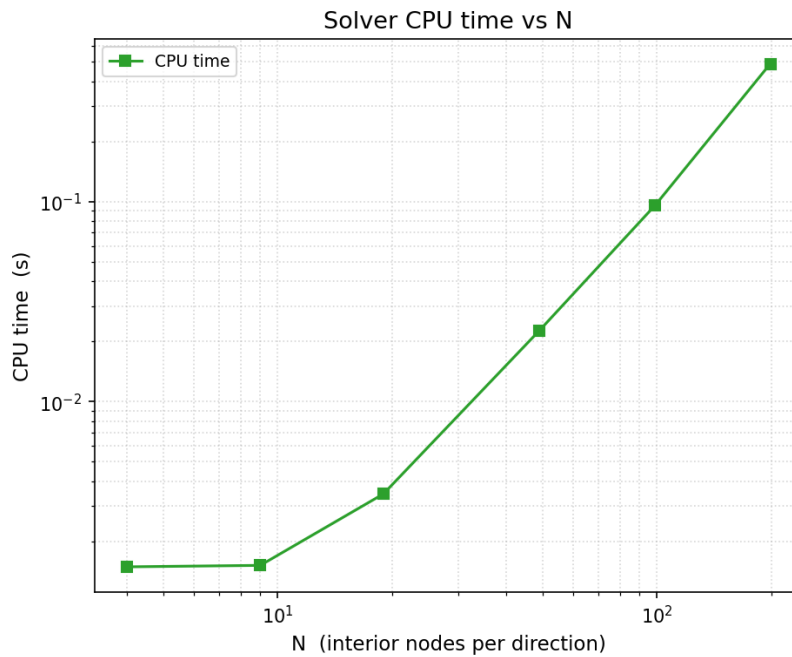


FIGURE 4 – Temps CPU du solveur en fonction de N (log-log). La courbe est **croissante**, ce qui est attendu.

Remarque 7.1 (Pourquoi le temps CPU est-il croissant ?). La courbe est **croissante avec N** , ce qui est physiquement correct : résoudre un système de taille $N^2 \times N^2$ prend plus de temps quand N augmente. La complexité de SuperLU pour le Laplacien 2D est $O(N^3)$ au pire, soit $O(h^{-3})$.

Attention à l'axe des abscisses. Si la courbe est tracée en fonction de $h = 1/(N + 1)$, elle *semble* décroissante (car h et N sont inversement proportionnels). Il

faut toujours préciser explicitement la variable en abscisse. Dans ce projet, l'axe est N , et la courbe est bien croissante.

Conclusion

Ce projet a mis en œuvre l'ensemble de la chaîne numérique pour l'équation de Poisson 2D :

- La **discrétisation par différences finies à 5 points** est consistante d'ordre 2 et converge avec $\alpha \approx 2.02$, confirmé par régression.
- L'**assemblage CSR** et la **résolution SuperLU** garantissent une efficacité mémoire et temporelle adaptée aux grandes tailles de maillage.
- Le format **VTK** permet une visualisation 3D avancée dans Paraview (coloration scalaire, *warp by vector*).
- Le format **HDF5** offre un stockage hiérarchique compressé, lisible par ViTables, avec attributs de convergence directement dans le fichier.
- Les **GIFs animés** rendent la convergence intuitive : on voit visuellement l'erreur s'annuler au fur et à mesure que N augmente.
- L'ensemble du pipeline est reproductible en une commande : `make run`.

Perspectives.

1. Implémenter un solveur itératif (gradient conjugué préconditionné) pour atteindre une complexité $O(N^2)$.
2. Étendre à des géométries non rectangulaires ou à des conditions de bord mixtes.
3. Paralléliser avec MPI (`mpi4py`) pour exploiter les calculateurs HPC.

Références

- [1] S. Dumont, *Cours : Outils pour la visualisation*, M1 CHPS, UPVD, 2025-2026. <https://perso.univ-perp.fr/sdumont/>
- [2] R. J. LeVeque, *Finite Difference Methods for Ordinary and Partial Differential Equations*, SIAM, 2007.
- [3] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, 2003.
- [4] P. Virtanen *et al.*, *SciPy 1.0 : Fundamental Algorithms for Scientific Computing in Python*, Nature Methods, **17**, 261–272, 2020.
- [5] J. Ahrens, B. Geveci, C. Law, *ParaView : An End-User Tool for Large Data Visualization*, Visualization Handbook, Elsevier, 2005.
- [6] The HDF Group, *Hierarchical Data Format, version 5*, 2014. <https://www.hdfgroup.org/HDF5/>